

**Distributed Systems
Workshop 5****Semester II 2026
Oct, 01 2026
Prof. Francisco Hidrobo****Distributed Databases****Prerequisites**

- Docker Engine / Docker Desktop and Docker Compose available.
- Internet access the first time the mysql:8.0 image is pulled.
- A terminal and text editor.
- At least three free host ports: 3306, 3307, and 3308.

Activity 1 — Create the Docker topology

Create a new folder, then create docker-compose.yml. The first two nodes form the replication pair; mysql-shard2 is an independent node used for sharding.

services:

mysql-primary:

image: mysql:8.0

container_name: mysql-primary

environment:

MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: lab

command:

- --server-id=1

- --log-bin=mysql-bin

- --binlog-format=ROW

- --gtid-mode=ON

- --enforce-gtid-consistency=ON

- --log-replica-updates=ON

ports:

- "3306:3306"

mysql-replica:

image: mysql:8.0

container_name: mysql-replica

environment:

MYSQL_ROOT_PASSWORD: root

command:

- --server-id=2

- --log-bin=mysql-bin

- --binlog-format=ROW

- --gtid-mode=ON

- --enforce-gtid-consistency=ON

```
--read-only=ON
```

```
ports:
```

```
- "3307:3306"
```

```
depends_on:
```

```
- mysql-primary
```

```
mysql-shard2:
```

```
image: mysql:8.0
```

```
container_name: mysql-shard2
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD: root
```

```
MYSQL_DATABASE: users_shard2
```

```
ports:
```

```
- "3308:3306"
```

```
docker compose up -d
```

```
docker ps
```

Wait until the MySQL servers accept connections. Verify each node:

```
docker exec mysql-primary mysqladmin ping -uroot -proot
```

```
docker exec mysql-replica mysqladmin ping -uroot -proot
```

```
docker exec mysql-shard2 mysqladmin ping -uroot -proot
```

Activity 2 — Primary–replica replication

On the primary, create the replication account:

```
docker exec mysql-primary mysql -uroot -proot -e "
CREATE USER IF NOT EXISTS 'repl'@'%' IDENTIFIED BY 'repl';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
FLUSH PRIVILEGES;"
```

Configure the replica using the Docker service name as the source host:

```
docker exec mysql-replica mysql -uroot -proot -e "
```

```
STOP REPLICATION;
```

```
CHANGE REPLICATION SOURCE TO
```

```
SOURCE_HOST='mysql-primary',
```

```
SOURCE_USER='repl',
```

```
SOURCE_PASSWORD='repl',
```

```
SOURCE_AUTO_POSITION=1;
```

```
GET_SOURCE_PUBLIC_KEY=1;
```

```
START REPLICATION;
```

```
SHOW REPLICATION STATUS\G"
```

Verify that both Replica_IO_Running and Replica_SQL_Running report Yes. Then create test data on the primary and verify it on the replica:

```
docker exec mysql-primary mysql -uroot -proot lab -e "
```

```
CREATE TABLE IF NOT EXISTS replication_test (id INT PRIMARY KEY, note  
VARCHAR(80));
```

```
INSERT INTO replication_test VALUES (1,'created on primary');"
```

```
docker exec mysql-replica mysql -uroot -proot lab -e "SELECT * FROM replication_test;"
```

Question: Why is the replica configured as read-only? What would become difficult to reason about if both nodes accepted independent writes?

Activity 3 — Horizontal sharding

Use the primary as Shard 1 and mysql-shard2 as Shard 2. Create the same schema on both nodes.

```
docker exec mysql-primary mysql -uroot -proot -e "
```

```
CREATE DATABASE IF NOT EXISTS users_shard1;
```

```
CREATE TABLE IF NOT EXISTS users_shard1.users
```

```
(id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(80), city VARCHAR(50));"
```

```
docker exec mysql-shard2 mysql -uroot -proot -e "
```

```
CREATE DATABASE IF NOT EXISTS users_shard2;
```

```
CREATE TABLE IF NOT EXISTS users_shard2.users
```

```
(id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(80), city VARCHAR(50));"
```

Apply this rule: even IDs → Shard 1; odd IDs → Shard 2. Each group must invent at least six users (three per shard).

Example pattern only

Shard 1: INSERT ... VALUES (2,...), (4,...), (6,...);

Shard 2: INSERT ... VALUES (1,...), (3,...), (5,...);

Run queries on each fragment separately and combine the two results in your report. Since these are different MySQL servers, it is not possible in this topology to perform a single local UNION operation that spans both servers; you act as the coordinator and reconstruct the logical set of users from the results of both queries.

Challenge: Propose a vertical sharding design for the same user data (for example, identity fields on one node and contact/profile fields on another). Try to implement and test it.

Activity 4 — Lost update and locking

Create the account table on the primary:

```
docker exec mysql-primary mysql -uroot -proot lab -e "
DROP TABLE IF EXISTS accounts;
CREATE TABLE accounts (
  id INT PRIMARY KEY, owner VARCHAR(50), balance INT NOT NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP
) ENGINE=InnoDB;
INSERT INTO accounts VALUES (1,'Alice',1000,NOW()),(2,'Bob',1000,NOW());"
```

Open two terminals connected to the primary:

```
docker exec -it mysql-primary mysql -uroot -proot lab
# Run the same command in a second terminal.
```

4.1 Reproduce the lost update

Coordinate the two sessions so both read 1000 before either application-level calculation is finalized. Use literal replacement values to mimic a naïve read–modify–write application.

SESSION A

```
SET SESSION TRANSACTION
ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT balance FROM accounts
WHERE id=1;
-- computes 1000 + 100
UPDATE accounts SET balance=1100
WHERE id=1;
COMMIT;
```

SESSION B

```
SET SESSION TRANSACTION
ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT balance FROM accounts
WHERE id=1;
-- computes 1000 - 50
UPDATE accounts SET balance=950
WHERE id=1;
COMMIT;
```

Record the final value and explain why the mathematically expected value, 1050, was not preserved.

4.2 Fix with SELECT ... FOR UPDATE

```
docker exec mysql-primary mysql -uroot -proot lab -e "UPDATE accounts SET balance=1000
WHERE id=1;"
```

Session A starts first and keeps the row locked. Session B should block at SELECT ... FOR UPDATE until A commits.

SESSION A

```
START TRANSACTION;
SELECT balance FROM accounts
WHERE id=1 FOR UPDATE;
UPDATE accounts SET balance =
balance + 100 WHERE id=1;
COMMIT;
```

SESSION B

```
START TRANSACTION;
SELECT balance FROM accounts
WHERE id=1 FOR UPDATE;
-- waits, then sees current row
UPDATE accounts SET balance =
balance - 50 WHERE id=1;
COMMIT;
```

```
docker exec mysql-primary mysql -uroot -proot lab -e "SELECT id,balance FROM accounts
WHERE id=1;"
```

Expected final balance: 1050. Evidence: screenshot showing Session B waiting and the final balance.

Activity 5 — Read-after-write consistency

Perform a write on the primary and immediately read the same row from the replica. Replication is asynchronous, so a stale read may be observable; on a fast local machine it may also be too brief to catch. Either outcome is valid if documented.

```
docker exec mysql-primary mysql -uroot -proot lab -e "
UPDATE accounts SET balance=balance+1 WHERE id=1;
SELECT id,balance,updated_at FROM accounts WHERE id=1;"
```

```
docker exec mysql-replica mysql -uroot -proot lab -e "SELECT id,balance,updated_at FROM
accounts WHERE id=1;"
```

For a read that must wait for the replica to catch up, capture the primary GTID set:

```
docker exec mysql-primary mysql -uroot -proot -e "SELECT @@GLOBAL.gtid_executed\G"
```

Copy the returned GTID set and use it on the replica:

```
docker exec -it mysql-replica mysql -uroot -proot
SELECT WAIT_FOR_EXECUTED_GTID_SET('<PASTE_GTID_SET_HERE>',5);
USE lab;
SELECT id,balance,updated_at FROM accounts WHERE id=1;
```

Record whether WAIT_FOR_EXECUTED_GTID_SET returns 0 (set applied) or 1 (timeout), and explain how this supports a read-your-writes requirement.

Activity 6 — Deadlock and prevention

```
docker exec mysql-primary mysql -uroot -proot lab -e "UPDATE accounts SET balance=1000
WHERE id IN (1,2);"
```

In two interactive sessions, acquire the two rows in opposite orders. Coordinate manually: A updates row 1; B updates row 2; then each tries to update the row held by the other.

SESSION A
START TRANSACTION;
UPDATE accounts SET balance=
balance+10 WHERE id=1;
-- wait for B first update
UPDATE accounts SET balance=
balance-10 WHERE id=2;

SESSION B
START TRANSACTION;
UPDATE accounts SET balance=
balance-10 WHERE id=2;
-- wait for A first update
UPDATE accounts SET balance=
balance+10 WHERE id=1;

One transaction should be selected as the deadlock victim and receive ERROR 1213. ROLLBACK or restart as needed.

6.2 Retry with consistent locking

-- In BOTH sessions, acquire the rows in the same order first:
START TRANSACTION;
SELECT * FROM accounts WHERE id IN (1,2) ORDER BY id FOR UPDATE;
-- then perform that session's updates
COMMIT;

Observe that one session may wait, but the circular wait is avoided. Explain the difference between blocking and deadlock.

Report

Keep the report concise (recommended 5–6 pages). It must contain:

- Group members
- Docker topology: primary, replica, and second shard.
- Replication evidence: status and one row successfully replicated.
- Sharding evidence: data on both shards and the reconstructed logical result.
- Lost-update evidence and explanation; locking fix with final balance 1050.
- Read-after-write observation and GTID-wait result.
- Deadlock evidence (ERROR 1213) and successful ordered-locking retry.
- A short discussion: how replication, fragmentation, and concurrency control address different problems in a distributed database.